Programación Orientada a Objetos

Paradigma de Programación POO

¿Qué es la Programación Orientada a Objetos?

La **Programación Orientada a Objetos (POO)** es un paradigma de programación que organiza el diseño de software alrededor de los **objetos** en lugar de las funciones y la lógica.

- Busca simular el mundo real a través de entidades de software, haciendo el código más intuitivo y cercano a la forma en que pensamos.
- **Ventajas principales:**
  - **Modularidad:** Divide el programa en módulos más pequeños y manejables.
  - **Reutilización de código:** Permite usar componentes existentes en nuevos programas.
  - **Facilidad de mantenimiento:** Los cambios en una parte del código tienen menos impacto en otras.
  - **Escalabilidad:** Facilita la expansión de las aplicaciones.

Programación Orientada a Objetos

Clases y Objetos

Clases

Una **clase** es un **plano o plantilla** para crear objetos. Define las **propiedades (atributos)** y **comportamientos (métodos)** que todos los objetos de esa clase tendrán.

- **Ejemplo del mundo real:** La clase Coche podría definir que todo coche tiene un color, marca y modelo, y puede acelerar o frenar.

```
// Definición de la clase Coche
class Coche {
public:
    // Atributos
    std::string marca;
    std::string modelo;
    int anio;

    // Métodos
    void acelerar() {
        // Implementación para acelerar
        std::cout << "El coche esta acelerando." << std::endl;
    }

    void frenar() {
        // Implementación para frenar
        std::cout << "El coche esta frenando." << std::endl;
    }
};
```

Programación Orientada a Objetos

Clases y Objetos

Objetos

Un **objeto** es una **instancia concreta** de una clase. Es una entidad real que se crea a partir de la plantilla de la clase y tiene un estado (valores específicos para sus atributos) y un comportamiento.

- **Ejemplo del mundo real:** miCocheFord es un objeto de la clase Coche con marca = "Ford", modelo = "Focus", año = 2020.

```
#include <iostream>
#include <string>
// (Clase Coche definida previamente)
int main() {
    // Creación de objetos de la clase Coche
    Coche miCocheFord; // Objeto 1
    Coche tuCocheHonda; // Objeto 2

    // Asignando valores a los atributos del objeto1
    miCocheFord.marca = "Ford";
    miCocheFord.modelo = "Focus";
    miCocheFord.anio = 2020;

    // Accediendo a los métodos del objeto miCocheFord
    std::cout << "Mi coche es un " << miCocheFord.marca << " "
<< miCocheFord.modelo << std::endl;
    miCocheFord.acelerar();

    return 0;
}
```

Programación Orientada a Objetos

Sintaxis y Semántica

- **Sintaxis:** Se refiere a las **reglas gramaticales** de un lenguaje de programación. Dicta cómo se deben escribir las instrucciones, clases, métodos, etc., para que el compilador las entienda y no genere errores.
  - **Ejemplo en C++:** El uso de punto y coma (;) al final de una sentencia o la palabra clave class para definir una clase.
- **Semántica:** Se refiere al **significado** de las instrucciones. Describe qué acción realiza un fragmento de código cuando se ejecuta.
  - **Ejemplo en C++:** La semántica del método acelerar() en la clase Coche es aumentar la velocidad del vehículo.

Programación Orientada a Objetos

Tipos Abstractos de Datos (TAD)

Un **Tipo Abstracto de Datos (TAD)** es un concepto que define un conjunto de datos y las operaciones que se pueden realizar sobre ellos, **sin especificar cómo se implementan internamente**. Se enfoca en el "que" se puede hacer, no en el "cómo" se hace.

- Una **clase en POO** es una forma de implementar un TAD.

Propiedades de los Tipos Abstractos de Datos

1. **Ocultamiento de Información:** La implementación interna de los datos y las operaciones está oculta al usuario o a otras partes del programa.
2. **Especificación Formal:** Se define la interfaz del TAD (operaciones y su comportamiento) de manera precisa y formal.
3. **Independencia de Implementación:** Permite cambiar la implementación interna del TAD sin afectar el código que lo usa, siempre que la interfaz pública se mantenga.

Programación Orientada a Objetos

Abstracción

La **Abstracción** es el proceso de **identificar las características esenciales** de un objeto e ignorar los detalles irrelevantes o menos importantes. Permite modelar problemas complejos reduciendo su complejidad.

- En POO, la abstracción se logra principalmente a través de las **clases**, que encapsulan los atributos y métodos relevantes de una entidad del mundo real.
- **Ejemplo:** Al modelar un Cliente en un sistema bancario, nos interesa su nombre, número de cuenta y saldo, no su altura o color de pelo.

Programación Orientada a Objetos

Encapsulamiento

El **Encapsulamiento** es el mecanismo de la POO que **combina los datos (atributos) y las funciones (métodos) que operan sobre esos datos en una única unidad (la clase)**. Además, **restringe el acceso directo a algunos de los componentes del objeto** para proteger su integridad.

- **Beneficios:**
  - **Protege la integridad de los datos:** Controla cómo se accede y se modifican los atributos.
  - **Reduce la complejidad:** Los usuarios solo necesitan interactuar con la interfaz pública.
  - **Mejora la modularidad:** Permite modificar la implementación interna sin afectar el código externo.
- Se logra a través de los **especificadores de acceso** en C++.

Programación Orientada a Objetos

Orientación a Objetos en C++

Atributos y Métodos

- **Atributos (Variables miembro):** Son variables declaradas dentro de una clase que representan las características o el estado de un objeto.
    - **Ejemplo:** nombre y edad en la clase Persona.
- **Métodos (Funciones miembro):** Son funciones declaradas dentro de una clase que definen las operaciones o el comportamiento que puede realizar un objeto.
    - **Ejemplo:** saludar() en la clase Persona.

```
class Persona {
public: // Miembros públicos
    std::string nombre;
    int edad;

    void saludar() {
        std::cout << "Hola, mi nombre es " << nombre <<
std::endl;
    }
};
```

    -

Programación Orientada a Objetos

Creación de Objetos - Crear un objeto es declarar una instancia de una clase.

```
#include <iostream>
#include <string>

// (Clase Persona definida previamente)

int main() {
    // Creación de un objeto en el stack (automático)
    Persona persona1;

    // Asigna valores a los atributos de la clase
    persona1.nombre = "Alice";
    persona1.edad = 30;
    persona1.saludar();

    // Creación de un objeto en el heap (memoria dinámica)
    // Se usa 'new' para asignar memoria y 'delete' para liberarla
    Persona* persona2 = new Persona(); // Puntero a la clase
    persona2->nombre = "Bob"; // Se usa '->' para acceder a miembros de punteros
    persona2->edad = 25;
    persona2->saludar();

    delete persona2; // Liberar la memoria asignada dinámicamente

    return 0;
}
```

Programación Orientada a Objetos

Mensajes

En POO, los **mensajes** son la forma en que los objetos **interactúan entre sí**. Un objeto envía un mensaje a otro objeto invocando uno de sus métodos.

- **Sintaxis:** objeto.metodo(argumentos);
- **Ejemplo:**

// Instancia Clase Coche

```
Coche miCoche;
miCoche.acelerar(); // Enviando el mensaje "acelerar" al objeto miCoche
```

Programación Orientada a Objetos

Especificadores de Acceso

Los **especificadores de acceso** controlan la visibilidad y el acceso a los miembros (atributos y métodos) de una clase, implementando el **encapsulamiento**.

- **public:** Los miembros declarados como public son **accesibles desde cualquier parte** del programa. Constituyen la interfaz pública de la clase.
- **private:** Los miembros declarados como private solo son **accesibles desde dentro de la propia clase**. Son internos a la implementación y no pueden ser accedidos directamente desde fuera.
- **protected:** Los miembros declarados como protected son accesibles desde la propia clase y desde las **clases derivadas (herencia)**.

Programación Orientada a Objetos

Especificadores de Acceso

```
class CuentaBancaria {
private: // Atributo privado, solo accesible desde métodos de CuentaBancaria
    double saldo;

public: // Métodos públicos para interactuar con el saldo
    CuentaBancaria(double s) : saldo(s) {} // Constructor

    void depositar(double cantidad) {
        if (cantidad > 0) {
            saldo += cantidad;
            std::cout << "Deposito de " << cantidad << ". Nuevo saldo: " << saldo << std::endl;
        }
    }

    void retirar(double cantidad) {
        if (cantidad > 0 && saldo >= cantidad) {
            saldo -= cantidad;
            std::cout << "Retiro de " << cantidad << ". Nuevo saldo: " << saldo << std::endl;
        } else {
            std::cout << "No se pudo realizar el retiro." << std::endl;
        }
    }

    double getSaldo() { // Método para obtener el saldo (interfaz pública)
        return saldo;
    }
};
```

Programación Orientada a Objetos

Especificadores de Acceso

```
// Trabajar en la función main con objetos


int main() {
      // Instanciar objeto CuentaBancaria inicializando sus atributos
      CuentaBancaria miCuenta(1000.0);
      // Invoca al método depositar para modificar el valor del atributo
      miCuenta.depositar(200.0);
      // Invoca al método depositar para modificar el valor del atributo
      miCuenta.retirar(500.0);

      // Referenciar en forma directa a un atributo “privado” es un ERROR
      // miCuenta.saldo = 5000.0; // ERROR: 'saldo' es privado
      // Invocar al método getSaldo() para acceder al valor actual del atributo privado saldo
      std::cout << "Saldo actual: " << miCuenta.getSaldo() << std::endl;
    return 0;
}
```

Programación Orientada a Objetos

Constructores

Los **constructores** son métodos especiales de una clase que se **ejecutan automáticamente cuando se crea un objeto** de esa clase. Su principal propósito es **inicializar los atributos** del objeto para asegurar un estado válido.

- Tienen el **mismo nombre que la clase**.
- **No tienen tipo de retorno** (ni siquiera void).

Tipos de Constructores

1. **Constructor por Defecto:** No toma argumentos. Si no defines uno, C++ genera uno por ti (si no hay otros constructores definidos).
2. **Constructor Parametrizado:** Toma uno o más argumentos para inicializar los atributos con valores específicos.
3. **Constructor de Copia:** Se usa para crear un nuevo objeto como una copia exacta de un objeto existente.

Programación Orientada a Objetos

Constructores - Ejemplo Default

```
class MiClase {
public:
    MiClase() { // Constructor por defecto
        std::cout << "Constructor por defecto llamado." << std::endl;
    }
};
int main(){
      // Uso: MiClase obj;
      MiClase obj;
      return 0;
}
```

Programación Orientada a Objetos

Constructores - Ejemplo Parametrizado

```
class MiClaseConParam {
public:
    int valor;
    MiClaseConParam(int v) { // Constructor parametrizado
        valor = v;
        std::cout << "Constructor parametrizado llamado con valor: " << valor << std::endl;
    }
};
int main(){
     // Uso: MiClaseConParam obj(10);
     MiClaseConParam obj(10);
     return 0;
}
```

Programación Orientada a Objetos

Constructores - Ejemplo Copia

```
class MiClaseCopia {
public:
    int id;
    MiClaseCopia(int i) : id(i) {} // Constructor normal

    MiClaseCopia(const MiClaseCopia& otro) { // Constructor de copia
        id = otro.id;
        std::cout << "Constructor de copia llamado. ID: " << id << std::endl;
    }
};
int main(){
     // Uso: MiClaseCopia obj1(5); MiClaseCopia obj2 = obj1;
     MiClaseCopia obj1(5);
     MiClaseCopia obj2 = obj1;
     return 0;
}
```

Programación Orientada a Objetos

Destructores

Un **destructor** es un método especial de una clase que se **ejecuta automáticamente cuando un objeto es destruido** (es decir, cuando su tiempo de vida finaliza o cuando se libera su memoria).

- Su objetivo es **liberar los recursos** que el objeto haya podido adquirir durante su vida útil (ej. memoria dinámica, archivos abiertos, conexiones de red).
- Tienen el **mismo nombre que la clase, precedido por una tilde (~)**.
- **No toman argumentos** y **no tienen tipo de retorno**.
- Solo puede haber **un destructor** por clase.

Programación Orientada a Objetos

Destructores - Ejemplo

```
#include <iostream>

class Recurso {
public:
    Recurso() {
        std::cout << "Recurso adquirido (objeto creado)." << std::endl;
        // Simular asignación de memoria dinámica
        int* ptr = new int[10];
    }
    ~Recurso() { // Destructor
        std::cout << "Recurso liberado (objeto destruido)." << std::endl;
        // Simular liberación de memoria dinámica
        // delete[] ptr; // (Necesitaríamos guardar ptr como un atributo para liberarlo)
    }
};
int main() {
    std::cout << "Inicio del main." << std::endl;
    Recurso obj1; // El destructor de obj1 se llamará al salir del scope
    {
        Recurso obj2; // El destructor de obj2 se llamará al salir de este bloque
    } // obj2 es destruido aquí
    std::cout << "Fin del main." << std::endl;
    return 0;
} // obj1 es destruido aquí
```